Relatório de Implementação: Projeto BuildFit

Este documento detalha o cumprimento dos requisitos propostos para a etapa de estruturação básica da aplicação web **BuildFit**. Foram implementadas as interfaces visuais, organização de templates, definição estilística e roteamento principal do sistema utilizando o ecossistema Node.js.

1. Planejamento do Modelo Básico (Layout)

O layout do BuildFit foi planejado com foco na fluidez e responsividade (abordagem *fluid*). As telas de formulário (Login e Cadastro) utilizam um modelo centralizado e fixo em largura máxima para manter a legibilidade. Já a tela principal de construtor de treinos utiliza um modelo de **duas colunas fluidas** (implementado via sistema de grid): a coluna da esquerda abriga a biblioteca de componentes (exercícios e treinos) e a coluna da direita atua como a área de trabalho (cronograma e tabelas de preenchimento).

2. Paleta de Cores

A identidade visual do projeto segue uma estética moderna inspirada em aplicativos *fitness* de alto desempenho ("Dark Mode" nativo com elementos de destaque em cores quentes).

- **Fundo principal:** Escuro profundo (`#0d0d0d`), reduzindo o cansaço visual.
- **Cards e containers (Glassmorphism):** Cinza escuro (`#1e1e1e`) com bordas sutis.
- **Destaque e Call-to-Action:** Degradê linear transitando de rosa carmim (`#ff416c`) para laranja vibrante (`#ff4b2b`). Utilizado em botões e títulos principais.

3. Tipografia e Fontes

A fonte escolhida para todo o projeto foi a **Inter**, uma família tipográfica sem serifa (*sans-serif*) altamente legível e projetada especificamente para interfaces de usuário em telas de computador. Ela foi importada via Google Fonts e definida globalmente no CSS como a fonte primária, provendo excelente clareza tanto em títulos grandes (3rem) quanto em dados tabulares menores.

4. Definição do Framework CSS

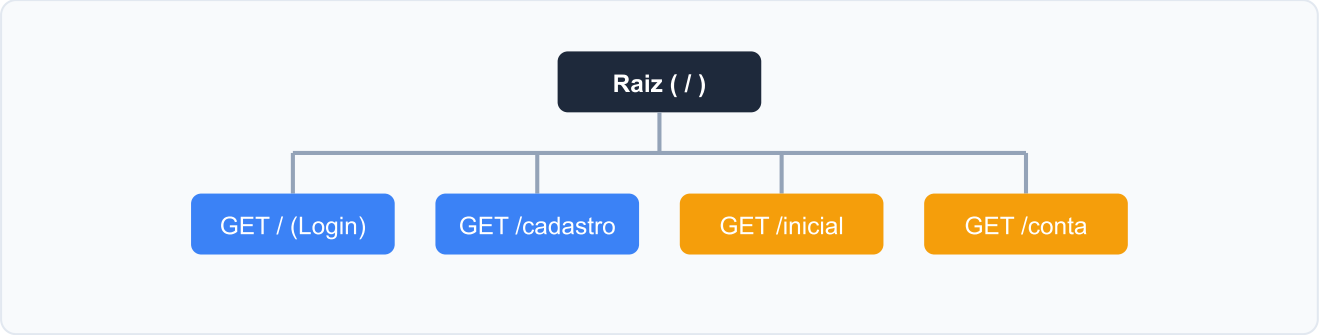
Para acelerar o desenvolvimento do layout, garantir a responsividade e utilizar componentes padronizados, adotou-se o **Bootstrap 5** via CDN. O Bootstrap gerencia o sistema de grid (classes `col-md` , `row`), o espaçamento utilitário (`mt-4` , `p-3`) e a base estrutural das tabelas e formulários. A customização visual profunda (cores, degradês, cursores e pop-ups) foi sobreposta com CSS puro, respeitando a hierarquia do framework.

5. Criação do HTML Base Integrado ao CSS

A estruturação visual foi consolidada com a criação de páginas base integrando o framework e o estilo customizado (`style.css`). Formulários estilizados com campos escuros, bordas secundárias e botões gradientes foram aplicados. Os *placeholders* foram tratados para oferecer boa visibilidade contra o fundo escuro, garantindo uma interface rica e pronta para interações.

6. Diagrama de Árvore Invertida (Roteamento)

O sistema foi desenhado a partir de uma rota raiz que distribui o acesso para as demais áreas públicas e privadas do sistema. Abaixo, a representação visual da topologia de rotas construída:



7. Criação do Projeto e IDE

A implementação teve início a partir da inicialização de um pacote Node (`npm init`) gerenciado através do **Visual Studio Code (VS Code)**, a IDE escolhida para o desenvolvimento. A infraestrutura de pastas foi segmentada separando claramente a lógica de backend (raiz), arquivos estáticos em `/public` (CSS, imagens e JS do cliente) e as visualizações em `/views` .

8. Infraestrutura Básica com Node.js e Express

O alicerce da aplicação foi estabelecido no arquivo principal `app.js`, instanciando um servidor web utilizando o framework **Express**. Foram configurados os *middlewares* necessários para tratar dados codificados na URL e arquivos JSON (`express.urlencoded` e `express.json`), e foi liberado o acesso à pasta de ativos estáticos.

Nesta camada, todas as rotas desenhadas no diagrama do Passo 6 foram mapeadas e interligadas às suas respectivas respostas, garantindo a navegação plena entre Login, Cadastro, Inicial e Conta, mesmo sem a integração final com bancos de dados relacionais estipulada para a fase N2.

9. Implantação de Template Engine (EJS)

Para renderizar o HTML dinamicamente e obedecer ao requisito de organização visual do Passo 5, foi configurado o **EJS (Embedded JavaScript templating)** como o motor de visualização oficial da aplicação (`app.set('view engine', 'ejs')`). Todos os documentos HTML estáticos gerados inicialmente foram convertidos para a extensão `.ejs`, permitindo futura injeção de dados vindos do servidor.

10. Utilização de Templates com Partials

A fim de facilitar a manutenção estrutural do projeto e evitar a repetição de código no cabeçalho das páginas (onde são importados o Bootstrap, as fontes e os arquivos CSS estáticos), foi criada a arquitetura de **Partials**. O cabeçalho foi isolado no arquivo `views/partials/head.ejs`. Dessa forma, as quatro rotas principais do sistema iniciam seus arquivos invocando a diretiva modular:

```
<%- include('partials/head') %>
```

Isso unifica a gestão de dependências da interface, cumprindo integralmente o requisito de componentização de interface exigido no escopo da disciplina.